MyCodeSchool

# Code review — usermanagement-UI

Sebastian · runde de review pe proiectul tău, cea mai nouă sus.

---

## Runda 1 — 2026-11-12 — sha 894d226

Am revizuit fișa „User Management în Angular — fără server" (ex. 1–9 + bonus), la ultimul tău commit, „fisa". Proiectul compilează (`ng build` ✅). L-am rulat în browser: am completat formularul, am salvat și m-am uitat la tabel, la statistici și la consolă.

**Ce e bine:** interfața `User` stă în fișierul ei (`models/user.ts`). Lista e un `signal` cu 15 utilizatori. Statisticile din bonus (vârsta medie, minori, adulți) sunt `computed`. Inputurile sunt legate corect, cu `#ref` + `[value]` + `(input)`.

**Atenție la istoric:** în `deb8424` („finalizare tabel mockup") aveai `remove`, `edit` și un `save` cu adăugare + editare scrise corect, iar „Sterge" mergea (15 → 14). În `894d226` ai adăugat formularul, dar ai rescris `users.ts` de la zero și le-ai pierdut. Ex. 6 și 8 nu mai există, iar `save` s-a stricat (vezi B2).

### 🔴 Critice

**B1 🔴 „Salveaza" e activ când formularul e gol și inactiv când e completat** `users.html:52` · `users.ts:71–75`

CUM E ACUM: la pornire butonul e albastru. Apeși pe el și se adaugă un rând gol: `Utilizatori 16`, nume `" "`, `0 ani`. Completezi toate câmpurile și butonul devine gri.

DE CE: `[disabled]="canSave()"` copiază valoarea în proprietatea `disabled` a butonului, iar `disabled = true` înseamnă „nu se poate apăsa". `canSave()` e `true` exact atunci când datele sunt bune, deci butonul se dezactivează tocmai când ai voie să salvezi. Ai nevoie de opusul ei: `!canSave()`. În plus, `canSave` nu verifică vârsta, cum cere ex. 9.

FIX: `[disabled]="!canSave()"` și `&& this.age() > 0` în `canSave`.

**B2 🔴 După „Salveaza" statisticile rămân vechi** `users.ts:109–111`

CUM E ACUM: adaugi „Test Elev", 16 ani. Rândul apare în tabel, dar „Minori" rămâne 2 (trebuia 3), „Adulti" crește de la 13 la 14 (trebuia să rămână 13), iar vârsta medie rămâne 29.6.

DE CE: `this.users()` îți dă chiar lista din semnal, nu o copie. `use.push(...)` o modifică pe loc, iar `update(() => use)` pune înapoi **aceeași** listă. Semnalul compară lista veche cu cea nouă după referință ( `Object.is` ) și vede aceeași adresă, deci consideră că nu s-a schimbat nimic. Pe cine depinde de el ( `averageAge` , `minori` ) nu-l anunță, iar ele păstrează valorile din cache. Tabelul s-a actualizat doar pentru că click-ul a redesenat componenta, iar `@for` a parcurs lista modificată. „Adulti" se calculează direct în template ( `users().length - minori()` ), de aceea crește greșit.

FIX: o listă **nouă**, `this.users.update((list) => [...list, user])` , ca în fișă.

### B3 🔴 Utilizatorii noi primesc un `id` care există deja `users.ts:65`

CUM E ACUM: primul utilizator nou primește `id: 6` , la fel ca Elena Nistor, iar al doilea primește 7, ca Andrei. În consolă apare `NG0955: The provided track expression resulted in duplicated keys` .

DE CE: `nextId = 6` venea din fișă, unde lista avea 5 utilizatori, dar tu ai 15. `track user.id` îi spune lui Angular „id-ul identifică rândul". Cu două rânduri cu același id, Angular nu mai știe care e care, iar un `remove(6)` le-ar șterge pe amândouă ( `filter` scoate tot ce are id 6).

FIX: `nextId = 16` , adică următorul număr după cel mai mare id din listă.

### B4 🔴 Câmpurile nu se golesc după salvare `users.ts:98–112`

CUM E ACUM: după „Salveaza" în câmpuri rămân „Test", „Elev", „t@e.ro", „16". Mai apeși o dată și primești un duplicat.

DE CE: inputurile afișează ce e în semnal ( `[value]="firstname()"` ). Cât timp semnalul păstrează textul, inputul îl arată. Golești câmpul doar golind semnalul.

FIX: la finalul lui `save()` , `this.firstname.set('')` (la fel pentru `lastname` și `email` ) și `this.age.set(0)` .

### B5 🔴 Ex. 6 (Sterge) și ex. 8 (Editeaza) au dispărut `users.html:94–99` · `users.ts`

CUM E ACUM: coloana „Actiuni" e goală (0 butoane în tabel), butoanele sunt comentate, iar `remove` și `edit` nu mai există în `users.ts` . În `deb8424` erau scrise și „Sterge" mergea.

DE CE: când rescrii un fișier de la zero, ce nu copiezi înapoi se pierde. Git le păstrează: `git show deb8424:src/app/users/users.ts` îți arată varianta veche.

FIX: readu `remove` și `edit` din `deb8424` , decomentează butoanele și pune în `save()` ramura `editedId() === 0 ? adaug : înlocuiesc` .

### B6 🔴 Un `<tr>` în interiorul unui `<td>` `users.html:83–92`

CUM E ACUM: în pagină sunt 30 de `<tr>` pentru 15 utilizatori. Linia de sub fiecare rând se oprește înainte de coloana „Actiuni", iar celulele nu mai stau sub capul de tabel.

DE CE: un rând de tabel conține doar celule ( `<td>` ), iar un `<tr>` nu poate sta într-o celulă. Angular construiește elementele direct din template, fără să-ți corecteze HTML-ul ca browserul, deci rândul al doilea chiar ajunge în celulă și strică alinierea coloanelor. Pe 2026-10-29 aveai aceeași problemă cu un `<button>` pus direct în `<tr>` .

FIX: șterge `<td>` , `<tr>` și `<td>` de la liniile 83–86. Rămâne o singură celulă, cu badge-ul.

## 🟡 Importante

- **M1** `users.html:87` : badge-ul folosește `user.age <= 18` , dar `minori` ( `users.ts:90` ) numără `< 18` . Alexandru (18) e galben pe ecran, deși la statistici e numărat adult. Fișa cere `< 18` .
- **M2** `users.html:72–103` : lipsește `@empty` („Niciun utilizator”) de la ex. 4.
- **M3** `users.html:76` : avatarul arată „B A”, cu spațiu, pentru că între cele două `{{ }}` e un spațiu. Trebuie `{{ user.firstName[0] }}{{ user.lastName[0] }}` .

## 🟢 Cleanups

- **C1** `users.ts:106` : un `console.log(newUser)` uitat.
- **C2** `users.html:48` : `value="0"` și `[value]="age()"` pe același input. Rămâne doar `[value]` .
- **C3** `users.html:40, 44` : lipsesc spațiile ( `"form-input"#second` , `"(input)` ). Prettier e instalat: `npx prettier --write src` .
- **C4** Numele: `UsersData` pentru o componentă (mai clar `UserList` ), `firstname` în semnal față de `firstName` în model, `let use` (mai clar `list` ).
- **C5** `users.html:60` : câmpul de căutare e copiat din mockup, dar nu face nimic. Fă bonusul sau scoate-l.

## Before / After

| | Acum | Corect |
|---|---|---|
| B1 | `[disabled]="canSave()"` | `[disabled]="!canSave()"` |
| B2 | `let use = this.users(); use.push(newUser); this.users.update(() => use);` | `this.users.update((list) => [...list, newUser]);` |
| B3 | `nextId: number = 6;` | `nextId: number = 16;` |
| B4 | *(nimic după salvare)* | `this.firstname.set(''); … this.age.set(0);` |
| B5 | `<!-- <button (click)="remove(user.id)" …> -->` | `<button class="btn btn-danger" (click)="remove(user.id)">Sterge</button>` |
| B6 | `<td>` `<tr><td>@if …</td>` | `<td>@if …</td>` |

## Întrebări

1. Dacă în `save()` scrii `this.users.update((list) => { list.push(user); return list; })`, se actualizează „Minori”? De ce?
2. Ce se întâmplă cu `remove(6)` dacă două rânduri au `id` 6? Dar cu `track user.id`?
3. Ce valoare are `canSave()` când toate câmpurile sunt completate și ce face atunci `[disabled]` cu ea?